

Suricata IDS + Wazuh Integration on GCP

Complete Step-by-Step Guide for Beginners

Network Intrusion Detection • Threat Intelligence • MITRE ATT&CK; Mapping

*Author: Samson Ejim
Cybersecurity Enthusiast*

Google Cloud Platform | Ubuntu 22.04 LTS | Suricata 8.0.4 | Wazuh 4.7.5

■ Table of Contents

Introduction & Prerequisites	3
Architecture Overview	3
Detection Pipeline	4
Phase 1 — Install Suricata on Ubuntu Endpoint	4
Phase 2 — Configure Suricata (Network Interface & Rules)	5
Phase 3 — Configure Wazuh Agent to Read Suricata Logs	6
Phase 4 — Configure Wazuh Manager	7
Phase 5 — Simulate Attacks with Nmap	8
Phase 6 — Validate Alerts on Wazuh Dashboard	9
Understanding Your Alerts	10
MITRE ATT&CK; Mapping	10
Troubleshooting Common Errors	11
Summary & Next Steps	12

■ Introduction

This guide walks you through deploying **Suricata** — a powerful open-source Network Intrusion Detection System (IDS) — and integrating it with your existing **Wazuh SIEM** on Google Cloud Platform. While Wazuh's File Integrity Monitoring (FIM) watches what happens on disk, Suricata watches what happens on the **network** — catching threats before they ever touch the filesystem. Together they form a layered SOC detection stack used in real enterprise environments.

What You Will Need

Existing Wazuh Lab	wazuh-server and ubuntu-endpoint VMs from previous lab still running
GCP Console Access	To SSH into both VMs
VirusTotal Lab Complete	This lab builds directly on top of the previous project
Basic Linux knowledge	Ability to run commands in a terminal

■ TIP

This lab reuses your existing GCP VMs — no new infrastructure costs!

■ Architecture Overview

Suricata runs on the **ubuntu-endpoint** and monitors all traffic on the **ens4** network interface. When it detects suspicious traffic, it writes a JSON alert to **/var/log/suricata/eve.json**. The Wazuh Agent reads this file and forwards alerts to the Wazuh Manager, where they appear on the dashboard.

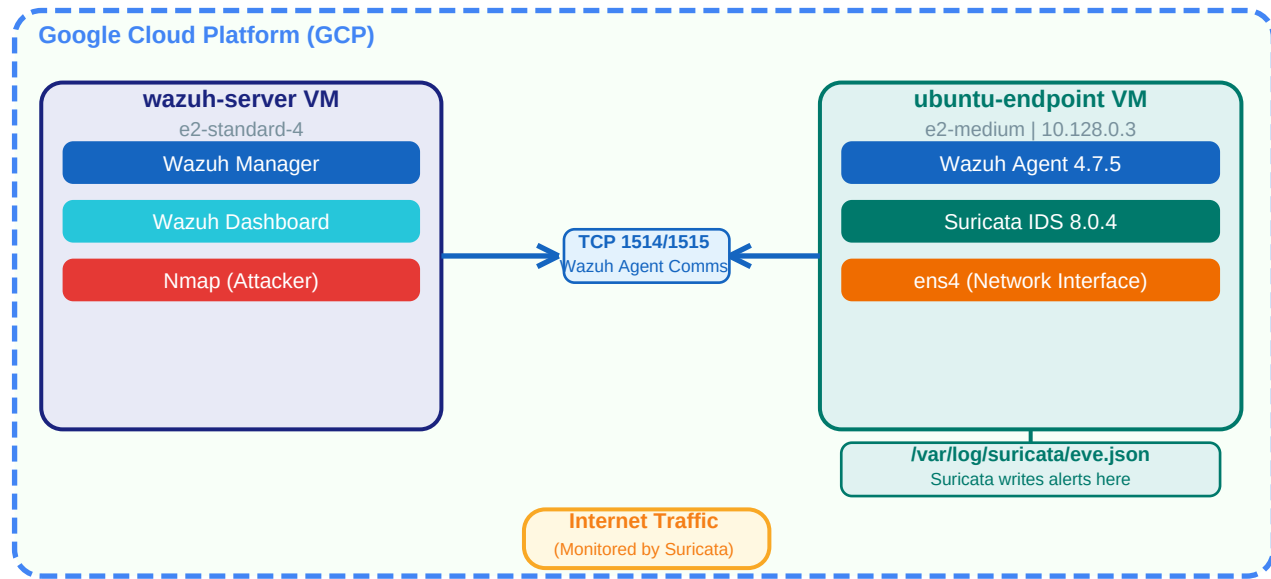


Figure 1: Suricata IDS + Wazuh Integration Architecture on GCP

■ Detection Pipeline

Every packet entering or leaving the endpoint is inspected by Suricata against **40,000+ community threat signatures**. When a match is found:

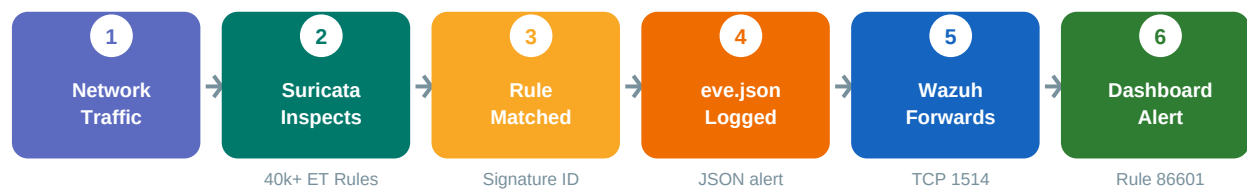


Figure 2: Network Traffic → Suricata → eve.json → Wazuh → Dashboard

PHASE 1 Install Suricata on Ubuntu Endpoint

SSH into your **ubuntu-endpoint** VM from GCP Console. Click the **SSH** button next to ubuntu-endpoint in Compute Engine → VM Instances.

1 Add the official Suricata repository

```
sudo add-apt-repository ppa:oisf/suricata-stable -y
sudo apt-get update
```

■ NOTE

The ppa:oisf/suricata-stable repository always provides the latest stable Suricata release.

2 Install Suricata

```
sudo apt-get install suricata -y
```

This installs Suricata and all its dependencies. Takes 1–2 minutes.

3 Verify installation

```
suricata -V
# Expected: This is Suricata version 8.0.4 RELEASE
```

4 Find your network interface name

Suricata needs to know which network interface to monitor. Run:

```
ip a
```

Look for the interface with your GCP internal IP (10.128.0.x). On GCP it is typically **ens4**. Note this name — you need it in Phase 2.

5 Check Suricata service status

```
sudo systemctl status suricata
```

■ WARNING

Suricata may show 'failed' at this stage — that is expected! It cannot start until we configure the network interface in Phase 2.

PHASE 2 Configure Suricata

Now configure Suricata to monitor the correct network interface and define which IP addresses are part of your protected network.

1 Open the Suricata configuration file

```
sudo nano /etc/suricata/suricata.yaml
```

■ NOTE

This is a large file. Use Ctrl+W to search for specific text instead of scrolling.

2 Set HOME_NET to your endpoint IP

Press **Ctrl+W**, type **HOME_NET** and press Enter. Find:

```
HOME_NET: "[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]"
```

Change it to your specific endpoint IP:

```
HOME_NET: "[10.128.0.3/32]"
```

■ NOTE

The /32 means exactly one IP address — your ubuntu-endpoint internal IP. Replace 10.128.0.3 with your actual IP if different.

3 Update EXTERNAL_NET

Just below HOME_NET, find:

```
EXTERNAL_NET: "!$HOME_NET"  
#EXTERNAL_NET: "any"
```

Swap the comments so it looks like this:

```
#EXTERNAL_NET: "!$HOME_NET"  
EXTERNAL_NET: "any"
```

■ TIP

Setting EXTERNAL_NET to 'any' makes Suricata treat ALL other traffic as potentially hostile — more aggressive detection for the lab.

4 Set the network interface

Press **Ctrl+W**, search for **af-packet**. Find:

```
af-packet:  
- interface: eth0
```

Change **eth0** to **ens4**:

```
af-packet:  
- interface: ens4
```

Save and exit: **Ctrl+X** → **Y** → **Enter**

5 Download threat detection rules

```
sudo suricata-update
```

This downloads 40,000+ signatures from the Emerging Threats community — one of the most comprehensive open-source threat intelligence feeds available. Takes 1–2 minutes.

6 Start Suricata

```
sudo systemctl restart suricata  
sudo systemctl status suricata  
# Expected: Active: active (running)
```

■ TIP

Verify Suricata is watching stats: `sudo tail -f /var/log/suricata/stats.log` — you should see counters updating every 8 seconds.

PHASE 3 Configure Wazuh Agent to Read Suricata Logs

The Wazuh Agent needs to know where Suricata saves its alerts. Suricata writes all alerts to `/var/log/suricata/eve.json` in JSON format. We add this path to the Wazuh Agent config so it monitors and forwards these alerts automatically.

1 Edit the Wazuh Agent config on ubuntu-endpoint

```
sudo nano /var/ossec/etc/ossec.conf
```

2 Add the Suricata log location

Press **Ctrl+End** to jump to the very bottom. Add this block just **above** the last `</ossec_config>` tag:

```
<localfile>
<log_format>json</log_format>
<location>/var/log/suricata/eve.json</location>
</localfile>
```

■ ■ WARNING

The `log_format` must be 'json' — this tells Wazuh to parse the `eve.json` file correctly as structured JSON data.

Save and exit: **Ctrl+X** → **Y** → **Enter**

3 Restart the Wazuh Agent

```
sudo systemctl restart wazuh-agent
sudo systemctl status wazuh-agent
# Expected: Active: active (running)
```

4 Verify Suricata is writing alerts

```
sudo tail -f /var/log/suricata/eve.json
# You should see JSON lines appearing
# Press Ctrl+C to stop
```

■ ■ NOTE

If `eve.json` is empty, wait 30 seconds and try again. Suricata starts writing as soon as it sees network traffic.

PHASE 4 Configure Wazuh Manager

SSH into **wazuh-server**. The Wazuh Manager already has built-in Suricata decoders and rules — we just need to verify they are present and restart the manager.

1 Verify built-in Suricata rules exist

```
sudo grep -r 'Suricata' /var/ossec/ruleset/rules/

# Expected output:
# /var/ossec/ruleset/rules/0475-suricata_rules.xml: Suricata rules
# /var/ossec/ruleset/rules/0475-suricata_rules.xml: Suricata Alert
# /var/ossec/ruleset/rules/0475-suricata_rules.xml: Suricata HTTP
# /var/ossec/ruleset/rules/0475-suricata_rules.xml: Suricata DNS
# /var/ossec/ruleset/rules/0475-suricata_rules.xml: Suricata TLS
```

TIP

Wazuh 4.7.x ships with built-in Suricata rules in file 0475-suricata_rules.xml — no manual rule writing needed!

2 Restart Wazuh Manager

```
sudo systemctl restart wazuh-manager
sudo systemctl status wazuh-manager
# Expected: Active: active (running)
```

3 Verify all Wazuh processes are running

Check the manager status output includes all these processes:

wazuh-integratord	Handles VirusTotal integration — still active from previous lab
wazuh-analysisd	Analyses and correlates all incoming alerts
wazuh-remoted	Receives data from Wazuh Agents
wazuh-logcollector	Collects logs from configured sources
wazuh-syscheckd	File Integrity Monitoring engine

PHASE 5 Simulate Attacks with Nmap

Now simulate real attacker reconnaissance from **wazuh-server** targeting **ubuntu-endpoint (10.128.0.3)**. Nmap is a standard network scanning tool used by both attackers and penetration testers.

■ WARNING

All scans are run within your own private GCP project against your own VMs. Never run these scans against systems you don't own.

1 Install Nmap on wazuh-server

```
sudo apt-get install nmap -y
```

2 SYN Stealth Scan — all 65,535 ports

```
sudo nmap -ss -p- 10.128.0.3
```

```
# What this does:  
# Sends SYN packets to every port without completing the TCP handshake  
# Classic attacker technique to discover open ports while evading basic detection
```

3 Aggressive Scan — OS fingerprinting + service detection

```
sudo nmap -A 10.128.0.3
```

```
# What this does:  
# Detects OS, software versions, runs NSE scripts  
# Very noisy — easily detected by IDS systems like Suricata
```

4 Additional evasion technique scans

```
# UDP scan  
sudo nmap -sU --top-ports 100 10.128.0.3  
  
# NULL scan — sends packets with no flags set  
sudo nmap -sN 10.128.0.3  
  
# XMAS scan — sends packets with FIN, PSF and URG flags  
sudo nmap -sX 10.128.0.3
```

■ NOTE

Wait 30-60 seconds after each scan before checking the dashboard — Wazuh needs time to process and display the alerts.

PHASE 6 Validate Alerts on Wazuh Dashboard

Open your Wazuh dashboard in a browser: https://<WAZUH_SERVER_EXTERNAL_IP> and verify the full pipeline.

1 Check Security Events for Suricata alerts

Go to **Modules** → **Security Events** and search for **suricata**

You should see alerts like these:

Rule ID	Alert Description	Threat Category
86601	ET DROP Dshield Block Listed Source	Known malicious IP blacklist
86601	ET CINS Active Threat Intelligence Poor Reputation IP	Threat intel blacklist
86601	ET DROP Spamhaus DROP Listed Traffic	Spam/malware source blacklist
86601	GPL RPC xdmcp info query	Remote desktop reconnaissance
86601	ET INFO GNU/Linux APT User-Agent Outbound	Package management traffic
5402	Successful sudo to ROOT executed	Privilege escalation

2 Check MITRE ATT&CK mapping

Click on any alert in the dashboard. Wazuh automatically maps alerts to the MITRE ATT&CK framework. You should see:

T1046	Network Service Discovery — Nmap port scanning detected
T1078	Valid Accounts — Authentication events flagged
T1548.003	Sudo and Sudo Caching — Privilege escalation detected

3 Verify Suricata stats on ubuntu-endpoint

```
sudo tail -f /var/log/suricata/stats.log
# You should see packet counters, memory usage, uptime

sudo grep '"event_type":"alert"' /var/log/suricata/eve.json | tail -5
# Shows the 5 most recent Suricata alerts in raw JSON
```

■ Understanding Your Alerts

Some of the alerts you see may be surprising — especially the blocklisted IP alerts that appear even without running Nmap. Here is what they mean:

ET DROP Dshield / Spamhaus / CINS Alerts

■ TIP

These fire when your GCP endpoint communicates with IP addresses on public threat intelligence blocklists. This is REAL traffic — not simulated. Cloud servers are constantly receiving unsolicited packets from scanners, bots, and malicious actors across the internet. Suricata catches these automatically.

GPL RPC xdmcp info query

■■ NOTE

This detects attempts to query your server's remote desktop configuration. Attackers use this to discover if remote desktop services are running and potentially exploitable.

ET INFO GNU/Linux APT User-Agent

■■ NOTE

This fires when your Ubuntu server runs apt-get to download packages. Suricata sees the outbound HTTP request with a Linux APT user-agent string. This is normal behaviour but Suricata flags it as informational — useful for tracking software installation activity.

■ MITRE ATT&CK; Mapping Explained

MITRE ATT&CK; is a globally recognised framework that categorises attacker techniques. Wazuh automatically maps alerts to this framework, giving SOC analysts instant context about what an attacker might be trying to do.

T1046 — Network Service Discovery	The attacker is mapping your network — finding open ports and services. Triggered by Nmap scans.
T1078 — Valid Accounts	Use of legitimate credentials to maintain access. Triggered by PAM login events.
T1548.003 — Sudo and Sudo Caching	Privilege escalation using sudo. Triggered when sudo commands are run on wazuh-server.

■ Troubleshooting Common Errors

Error 1: Suricata service fails to start

Symptom: Active: failed — Main process exited with status=1/FAILURE

Fix: The network interface or config is incorrect. Check that ens4 is set in suricata.yaml under af-packet.

```
sudo nano /etc/suricata/suricata.yaml
# Search for af-packet and confirm interface: ens4
sudo systemctl restart suricata
```

Error 2: No Suricata alerts appearing on Wazuh dashboard

Symptom: Security Events shows no suricata entries after running Nmap

Fix: The Wazuh Agent localfile config may be missing or incorrect. Verify eve.json path is added.

```
sudo grep -A3 'suricata' /var/ossec/etc/ossec.conf
# Should show: /var/log/suricata/eve.json
sudo systemctl restart wazuh-agent
```

Error 3: eve.json file is empty

Symptom: sudo tail -f /var/log/suricata/eve.json shows no output

Fix: Suricata may not be running or ens4 interface is wrong. Check Suricata status.

```
sudo systemctl status suricata
sudo cat /var/log/suricata/suricata.log | tail -20
# Look for errors about the network interface
```

Error 4: suricata-update fails

Symptom: Error downloading rules — network timeout or permission error

Fix: Check your internet connectivity and run with sudo.

```
sudo suricata-update
# If it still fails, try:
sudo suricata-update --no-reload
sudo systemctl restart suricata
```

■ Summary & What You Accomplished

Phase	Component	What was achieved
Phase 1	Install Suricata	Deployed Suricata 8.0.4 on ubuntu-endpoint via official OISF repository
Phase 2	Configure Suricata	Set HOME_NET, EXTERNAL_NET, ens4 interface + downloaded 40,000+ ET rules
Phase 3	Wazuh Agent Config	Added eve.json localfile monitor — Wazuh Agent now reads Suricata alerts
Phase 4	Wazuh Manager	Verified built-in Suricata decoder (0475-suricata_rules.xml) is active
Phase 5	Attack Simulation	Ran SYN, aggressive, UDP, NULL and XMAS Nmap scans from wazuh-server
Phase 6	Dashboard Validation	Confirmed Suricata alerts (Rule 86601) appearing with MITRE ATT&CK; mapping

■ Key Skills Demonstrated

- Network Intrusion Detection System (IDS) deployment and configuration
- Suricata traffic inspection using af-packet mode
- Emerging Threats community rule management with suricata-update
- SIEM log integration — forwarding eve.json alerts into Wazuh
- Attack simulation using Nmap (SYN, aggressive, UDP, NULL, XMAS scans)
- MITRE ATT&CK; framework alert mapping and interpretation
- Layered SOC detection — combining host-based (Wazuh FIM) + network-based (Suricata) monitoring
- Real threat intelligence — detecting actual blocklisted IP traffic on GCP

■ Full SOC Detection Stack

Project	Coverage	Repository
Wazuh FIM + VirusTotal	Host-based — file changes detected, malware confirmed, auto-deleted	github.com/samsonejim/wazuh-virustotal-soc-lab
Suricata IDS + Wazuh	Network-based — traffic inspected, threats detected, MITRE mapped	github.com/samsonejim/suricata-ids-wazuh-integration

Built with dedication by **Samson Ejim** — Cybersecurity Enthusiast
github.com/samsonejim | linkedin.com/in/samsonejim